

Td-Tp 6 : Réseau de Neurones Récurrent - Partie II

Dans ce devoir, vous allez implémenter le GRU et comparer les trois algorithmes (RNN, LSTM, GRU).

GRU

Le GRU (Gated Recurrent Unit) est une variante simplifiée du LSTM (Long Short-Term Memory), introduite par Cho et al. en 2014. Il vise à résoudre le problème du gradient qui disparaît dans les réseaux neuronaux récurrents classiques, tout en étant plus léger en paramètres que le LSTM.

Contrairement au LSTM qui utilise trois portes (entrée, oubli, sortie), le GRU n'en utilise que deux : la porte de mise à jour (update gate) et la porte de réinitialisation (reset gate).

On utilise les mêmes notations que pour la LSTM :

- x_t : vecteur d'entrée à l'instant t
- h_t : état caché à l'instant t
- y_t : sortie prédite à l'instant t
- W, b : paramètres appris (poids, biais)
- σ : fonction sigmoïde, $\tanh$: tangente hyperbolique

Table des matières

- [1 - Réseau à Unités Récurrentes à Portes \(GRU\)](#)
 - [1.1 Cellule GRU](#)
 - [Exercice 1 : gru_cell_forward](#)
 - [1.2 Propagation avant pour un réseau GRU complet](#)
 - [Exercice 2 : gru_forward](#)
 - [1.3 Rétropropagation pour un réseau GRU](#)
 - [Exercice 3 : gru_cell_backward](#)
 - [Exercice 4 : gru_backward](#)
- [2 - Comparaison entre RNN, LSTM et GRU pour le traitement de la langue française](#)
 - [2.1 RNN traditionnel](#)
 - [Exercice 5 : RNN class](#)
 - [2.2 LSTM](#)
 - [Exercice 6 : LSTM class](#)
 - [2.3 GRU](#)
 - [Exercice 7 : GRU class](#)
 - [2.4 Comparaison](#)

```
In [1]: import numpy as np
from rnn_utils import *
```

1 - Réseau à Unités Récurrentes à Portes (GRU)

Le GRU simplifie l'architecture LSTM tout en conservant un bon pouvoir de mémorisation. Il ne contient pas d'état de cellule séparé c_t : tout est contenu dans l'état caché h_t . On y trouve deux portes principales :

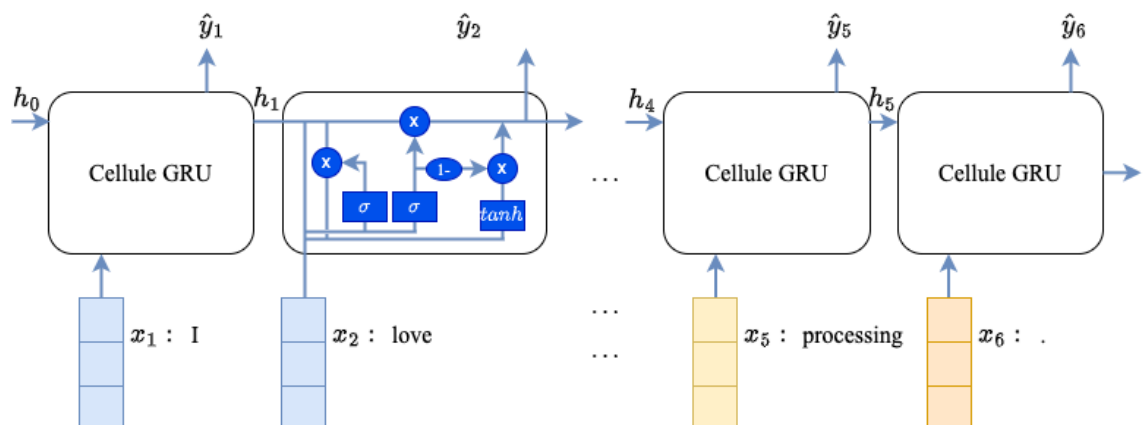


Figure 1 : Modèle basique de RNN

- Porte de mise à jour (update gate)

Cette porte décide combien de l'ancien état h_{t-1} on conserve :

$$(\Gamma_u)_t = \sigma_u (w_u^\top \cdot x_t + v_u^\top \cdot h_{t-1} + b_u) \quad (1)$$

- Porte de réinitialisation (reset gate)

Cette porte décide combien de l'ancien état h_{t-1} est ignoré lors de la création du nouveau contenu :

$$(\Gamma_r)_t = \sigma_r (w_r^\top \cdot x_t + v_r^\top \cdot h_{t-1} + b_r) \quad (2)$$

- État candidat

On calcule un nouvel état candidat $\tilde{h}_t$:

$$\tilde{h}_t = \tanh(w_h^\top \cdot x_t + v_h^\top \cdot ((\Gamma_r)_t \odot h_{t-1}) + b_h) \quad (3)$$

- État caché final

On interpole entre l'ancien état et l'état candidat avec $(\Gamma_u)_t$:

$$h_t = (\Gamma_u)_t \odot h_{t-1} + (1 - (\Gamma_u)_t) \odot \tilde{h}_t \quad (4)$$

- Sortie

Enfin, on calcule la sortie prédite :

$$\hat{y}_t = \text{softmax}(u_y^\top \cdot h_t + b_y) \quad (5)$$

1.1 - Cellule GRU

Exercice 1 Implémentez la cellule GRU décrite dans la Figure (1) d'après l'équation 1-5.

Instructions:

1. Concaténez w et v dans une matrice unique : $W = \begin{bmatrix} w \\ v \end{bmatrix}$
2. Concaténez x_t et h_{t-1} dans une matrice unique : $\text{concat} = \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix}$
3. Calculez toutes les formules 1-4. Vous pouvez utiliser `sigmoid()` (fourni) et `np.tanh()`. $\sigma_u = \text{sigmoid}$, $\sigma_r = \text{sigmoid}$, $\sigma_h = \tanh$.
4. Calculez la prédiction y_t . Vous pouvez utiliser `softmax()` (fourni).

```

In [36]: def gru_cell_forward(xt, h_prev, parameters):
        """
        Implémente une étape avant de la cellule GRU.
        Paramètres :
        xt -- Entrée au temps t, forme (d_x, n)
        h_prev -- État caché précédent, forme (d_h, n)
        parameters -- Dictionnaire contenant Wu, Wr, Wh, bu, br, bh, uy, by
        Retour :
        h_next -- Nouvel état caché
        yt_pred -- Prédiction au temps t
        cache -- Valeurs pour la rétropropagation
        """

        ## Récupération des paramètres
        # Wu, Wr, Wh, bu, br, bh, uy, by = ...
        ##### 1~8 lignes #####
        Wu, Wr, Wh = parameters['Wu'], parameters['Wr'], parameters['Wh']
        bu, br, bh = parameters['bu'], parameters['br'], parameters['bh']
        uy, by = parameters['uy'], parameters['by']
        #####

        ## Concaténation xt et h_prev
        # concat = ... (concat = [xt h_prev])
        ##### 1 ligne #####
        concat = np.vstack((xt, h_prev))
        #####

        ## Calcul des portes
        # gamma_u = ...
        # gamma_r = ...
        ##### 2 lignes #####
        gamma_u = sigmoid(np.dot(Wu, concat) + bu)
        gamma_r = sigmoid(np.dot(Wr, concat) + br)
        #####

        ## État du candidat
        # h_tilde = ...
        ##### 1 ligne #####
        h_tilde = np.tanh(np.dot(Wh, np.vstack((gamma_r * h_prev, xt))) + bh)
        #####

        ## Mise à jour état caché
        # h_next = ...
        ##### 1 ligne #####
        h_next = gamma_u * h_prev + (1 - gamma_u) * h_tilde
        #####

        ## Prédiction
        # yt_pred = ...
        ##### 1 ligne #####
        yt_pred = softmax(np.dot(uy, h_next) + by)
        #####

        cache = (h_next, h_prev, gamma_u, gamma_r, h_tilde, xt, parameters)
        return h_next, yt_pred, cache

```

```
In [37]: # Test
np.random.seed(1)
xt = np.random.randn(3, 10)
h_prev = np.random.randn(5, 10)
Wu = np.random.randn(5, 8)
Wr = np.random.randn(5, 8)
Wh = np.random.randn(5, 8)
uy = np.random.randn(2, 5)
parameters = {"Wu": Wu, "Wr": Wr, "Wh": Wh, "uy": uy, "bu": np.zeros((5,1)), "br": np.zeros((5,1))}
h_next, yt, cache = gru_cell_forward(xt, h_prev, parameters)
print("h_next[2] =", h_next[2])
print("h_next.shape =", h_next.shape)
print("yt[0] =", yt[0])
print("yt.shape =", yt.shape)
print("cache[1][3] =", cache[1][3])
print("len(cache) =", len(cache))
```

```
h_next[2] = [ 0.78201516 -0.40574735 -0.49169768 -0.20367101  0.57146426  0.15378341
 0.40015254  0.76039345  0.33266214  0.86245305]
h_next.shape = (5, 10)
yt[0] = [0.25248816 0.70157899 0.3381539  0.05242916 0.8491328  0.99070861
 0.47011064 0.04611104 0.42919554 0.01802984]
yt.shape = (2, 10)
cache[1][3] = [-0.75439794  1.25286816  0.51292982 -0.29809284  0.48851815 -0.07557171
 1.13162939  1.51981682  2.18557541 -1.39649634]
len(cache) = 7
```

Output attentus:

```
h_next[2] = [ 0.78201516 -0.40574735 -0.49169768 -0.20367101 0.57146426 0.15378341 0.40015254 0.76039345 0.33266214 0.86245305]
h_next.shape = (5, 10)
yt[0] = [0.25248816 0.70157899 0.3381539 0.05242916 0.8491328 0.99070861 0.47011064 0.04611104 0.42919554 0.01802984]
yt.shape = (2, 10)
cache[1][3] = [-0.75439794 1.25286816 0.51292982 -0.29809284 0.48851815 -0.07557171 1.13162939 1.51981682 2.18557541 -1.39649634]
len(cache) = 7
```

1.2 - Propagation avant pour un réseau GRU complet

Maintenant que nous avons implémenté une seule étape de la cellule GRU, nous pouvons l'utiliser dans une boucle pour traiter une séquence d'entrées avec T_x pas de temps. La propagation avant pour un réseau GRU complet consiste à appliquer la cellule GRU à chaque pas de temps et à collecter les états cachés et les prédictions.

Exercice 2 Implémentez la propagation avant pour un GRU.

Instructions:

1. Initialisez l'état caché h à zéro.
2. Utilisez une boucle `for` pour appliquer la cellule GRU à chaque pas de temps.
3. Stockez les états cachés et les prédictions dans des tableaux.

```
In [38]: def gru_forward(x, h0, parameters):
        """
        Propagation avant GRU sur T_x pas de temps.
        Paramètres :
        x -- Entrées de forme (d_x, n, T_x)
        h0 -- État initial (d_h, n)
        parameters -- Dictionnaire des paramètres
        Retour :
        h -- États cachés (d_h, n, T_x)
        y -- Prédictions (d_y, n, T_x)
        caches -- Informations pour la rétropropagation
        """

        ## Initialisation des paramètres
        # ... = x.shape
        # ... = h0.shape[0]
        # h = ...
        # y = ...
        # h_prev = ...
        ##### 5 lignes #####
        d_x, n, T_x = x.shape
        d_h = h0.shape[0]
        h = np.zeros((d_h, n, T_x))
        y = np.zeros((parameters['uy'].shape[0], n, T_x))
        h_prev = h0
        #####

        caches = []
        for t in range(T_x):
            ## Propagation avant
            # h_next, yt, cache = ...
            # h[:, :, t] = ...
            # y[:, :, t] = ...
            # caches
            # h_prev = ...
            ##### 5 lignes #####
            h_next, yt, cache = gru_cell_forward(x[:, :, t], h_prev, parameters)
            h[:, :, t] = h_next
            y[:, :, t] = yt
            caches.append(cache)
            h_prev = h_next
            #####
        caches = (caches, x)
        return h, y, caches
```

```
In [39]: # Test
x = np.random.randn(3,10,7)
h0 = np.random.randn(5,10)
h, y, caches = gru_forward(x, h0, parameters)
print("h[4][3] =", h[4][3])
print("h.shape =", h.shape)
print("y[1][4] =", y[1][4])
print("y.shape =", y.shape)
print("caches[1][1][3] =", caches[1][1][3])
print("len(caches) =", len(caches))

h[4][3] = [-0.30500452 -0.10598089  0.03128584  0.44701406  0.77709134  0.970725
 -0.52119401]
h.shape = (5, 10, 7)
y[1][4] = [0.18800902 0.6327917  0.79616097 0.62906616 0.03341917 0.20855483
 0.77001956]
y.shape = (2, 10, 7)
caches[1][1][3] = [-1.47115693 -0.8301719 -0.8805776 -0.27909772 1.62284909 0.01335268
 -0.6946936 ]
len(caches) = 2
```

Output attendus:

```
h[4][3] = [-0.30500452 -0.10598089 0.03128584 0.44701406 0.77709134 0.970725 -0.52119401]
h.shape = (5, 10, 7)
y[1][4] = [0.18800902 0.6327917 0.79616097 0.62906616 0.03341917 0.20855483 0.77001956]
y.shape = (2, 10, 7)
caches[1][1][3] = [-1.47115693 -0.8301719 -0.8805776 -0.27909772 1.62284909 0.01335268 -0.6946936 ]
len(caches) = 2
```

1.3 - Rétropropagation pour un réseau GRU

La rétropropagation pour un réseau GRU consiste à calculer les gradients de la fonction de coût par rapport aux paramètres du réseau. Cela se fait en parcourant la séquence en sens inverse et en appliquant la règle de la chaîne.

On suppose que l'on minimise une fonction de perte L dépendant de h_t . Notre but est de calculer les gradients

$$\frac{\partial L}{\partial W_u}, \frac{\partial L}{\partial W_r}, \frac{\partial L}{\partial W_h}$$

Étape 1 : Gradient de la perte par rapport à h_t C'est le signal de départ

$$dh_t = \frac{\partial L}{\partial h_t} \quad (6)$$

Étape 2 : Gradient par rapport aux composantes internes

- Gradient par rapport à $\tilde{h}_t$

$$d\tilde{h}_t = dh_t \odot (\Gamma_u)_t \quad (7)$$

- Puis, dérivée de $\tanh$

$$d \tanh = d\tilde{h}_t \odot (1 - \tilde{h}_t^2) \quad (8)$$

- Gradient par rapport à $(\Gamma_u)_t$

$$d(\Gamma_u)_t = d\tilde{h}_t \odot (h_{t-1} - \tilde{h}_t) \quad (9)$$

- Puis, dérivée de la sigmoïde σ_u

$$d\sigma_u = d(\Gamma_u)_t \odot (\Gamma_u)_t \odot (1 - (\Gamma_u)_t) \quad (10)$$

- Gradient par rapport à $(\Gamma_r)_t$ (à travers $\tilde{h}_t$)

$$d(\Gamma_r)_t = (d \tanh \cdot v_h) \odot h_{t-1} \quad (11)$$

$$d\sigma_r = d(\Gamma_r)_t \odot (\Gamma_r)_t \odot (1 - (\Gamma_r)_t) \quad (12)$$

Étape 3 : Rétropropagation aux poids Les dérivées par rapport aux poids (ex : w_u et v_u)

$$\frac{\partial L}{\partial w_u} = d\sigma_u \cdot x_t^T \quad (13)$$

$$\frac{\partial L}{\partial v_u} = d\sigma_u \cdot h_{t-1}^T \quad (14)$$

$$\frac{\partial L}{\partial b_u} = d\sigma_u \quad (15)$$

Exercice 3 Implémentez la cellule rétropropagation pour un réseau GRU d'après l'équation 6-15.

Instructions:

1. Calcul des gradients : La fonction calcule les gradients pour les portes de mise à jour (u_t), de réinitialisation (r_t), et de l'état candidat ($\tilde{h}_t$).
2. Gradients des entrées : Les gradients des entrées (dx_t) et de l'état caché précédent (dh_{prev}) sont calculés en utilisant les gradients des portes.
3. Gradients des poids et biais : Les gradients des matrices de poids (dW_u , dW_r , dW_h) et des biais (db_u , db_r , db_h) sont calculés en utilisant les gradients des portes et les entrées concaténées.


```

In [40]: def gru_cell_backward(dh_next, cache):
        """
        Calcule les gradients pour une cellule GRU à un pas de temps.

        Paramètres :
            dh_next -- Gradient de l'état caché suivant, forme (d_h, n)
            cache -- Tuple contenant (h_next, h_prev,  $\Gamma_u$ ,  $\Gamma_r$ , h_tilde, xt, paramètres)

        Retour :
            gradients -- Dictionnaire contenant :
                dxt : Gradient de l'entrée xt, forme (d_x, n)
                dh_prev : Gradient de l'état caché précédent, forme (d_h, n)
                dWu, dWr, dWh : Gradients des matrices de poids
                dbu, dbr, dbh : Gradients des biais
        """
        ## Récupération des valeurs du cache
        # ... = cache
        # ... = xt.shape
        # ... = h_next.shape[0]
        ##### 3 lignes #####
        h_next, h_prev, gamma_u, gamma_r, h_tilde, xt, parameters = cache
        d_x, n = xt.shape
        d_h = h_next.shape[0]
        #####

        ## Étape 1 : Calcul des gradients locaux (équations 1-3)
        # dsigma_u = ...
        # dtanh = ...
        # dsigma_r = ...
        ##### 3 lignes #####
        dsigma_u = dh_next * (h_prev - h_tilde) * gamma_u * (1 - gamma_u)
        dtanh = dh_next * gamma_u * (1 - h_tilde**2)
        dsigma_r = dtanh.dot(parameters['Wh'][:, :d_h].T) * h_prev * gamma_r * (1 - gamma_r)
        #####

        ## Étape 2 : Dérivées des matrices de poids (équations 4-6)
        # concat = ...
        # dWu = ...
        # dWr = ...
        # dWh = ...
        ##### 4 lignes #####
        concat = np.vstack((xt, h_prev)) # Concaténation [x_t, h_prev]
        dWu = np.dot(dsigma_u, concat.T)
        dWr = np.dot(dsigma_r, concat.T)
        dWh = np.dot(dh_tilde, np.vstack((gamma_r * h_prev, xt)).T)
        #####

        ## Étape 3 : Dérivées des biais (somme sur les échantillons)
        # dbz = ...
        # dbr = ...
        # dbh = ...
        ##### 3 lignes #####
        dbz = np.sum(dsigma_u, axis=1, keepdims=True)
        dbr = np.sum(dsigma_r, axis=1, keepdims=True)
        dbh = np.sum(dh_tilde, axis=1, keepdims=True)
        #####

        ## Étape 4 : Dérivées des entrées (équations 7-8)
        # dxt = ...
        # dh_prev = ...
        ##### ~2 lignes #####
        dxt = (
            np.dot(parameters['Wu'][:, d_x, :].T, dsigma_u) +
            np.dot(parameters['Wr'][:, d_x, :].T, dsigma_r) +
            np.dot(parameters['Wh'][:, d_x, :].T, dh_tilde)
        )
        dh_prev = (
            (1 - gamma_u) * dh_next +
            np.dot(parameters['Wu'][:, d_x, :].T, dsigma_u) +
            np.dot(parameters['Wr'][:, d_x, :].T, dsigma_r) +
            gamma_r * np.dot(parameters['Wh'][:, d_x, :].T, dh_tilde)
        )
        #####

        gradients = {
            "dxt": dxt, "dh_prev": dh_prev,
            "dWu": dWu, "dWr": dWr, "dWh": dWh,
            "dbu": dbu, "dbr": dbr, "dbh": dbh
        }

```



```
return gradients
```

```
In [43]: # Test
np.random.seed(1)
xt = np.random.randn(3, 10)
h_prev = np.random.randn(5, 10)

# Correction des noms de paramètres pour correspondre à l'implémentation
Wu = np.random.randn(5, 8) # Update gate weights
Wr = np.random.randn(5, 8) # Reset gate weights
Wh = np.random.randn(5, 8) # Candidate state weights
uy = np.random.randn(2, 5) # Output weights

parameters = {
    "Wu": Wu, "Wr": Wr, "Wh": Wh,
    "uy": uy,
    "bu": np.zeros((5,1)), # Correction du nom de biais
    "br": np.zeros((5,1)),
    "bh": np.zeros((5,1)),
    "by": np.zeros((2,1))
}

h_next, yt, cache = gru_cell_forward(xt, h_prev, parameters)
print("h_next.shape =", h_next.shape) # Doit être (5, 10)
print("yt.shape =", yt.shape) # Doit être (2, 10)
print("h_next[2] =", h_next[2][:3]) # 3 premières valeurs de la 3ème neurone
print("yt[0] =", yt[0][:3]) # 3 premières valeurs de la sortie
print("gamma_r[3][3] =", cache[3][3][3]) # Valeur spécifique pour vérification

h_next.shape = (5, 10)
yt.shape = (2, 10)
h_next[2] = [ 0.78201516 -0.40574735 -0.49169768]
yt[0] = [0.25248816 0.70157899 0.3381539 ]
gamma_r[3][3] = 0.04786229186201366
```

Output attendus:

```
h_next.shape = (5, 10)
yt.shape = (2, 10)
h_next[2] = [ 0.78201516 -0.40574735 -0.49169768]
yt[0] = [0.25248816 0.70157899 0.3381539 ]
gamma_r[3][3] = 0.04786229186201366
```

Étape 4 : Gradient à rétropropager à h_{t-1} On combine tous les chemins où h_{t-1} influence la sortie

$$dh_{t-1} = dh_t \cdot \frac{\partial h_t}{\partial h_{t-1}} = dh_t \odot (\Gamma_u)_t + (v_u^T \cdot d\sigma_u) + (v_r^T \cdot d\sigma_r) + ((\Gamma_r)_t \odot v_h^T \cdot d \tanh) \quad (16)$$

Exercice 4 Implémentez la rétropropagation pour un réseau GRU complet d'après l'équation 16.

Instructions:

1. Initialisez les gradients des paramètres.
2. Utilisez une boucle `for` pour appliquer la cellule GRU à chaque pas de temps.
3. Stockez les états cachés et les prédictions dans des tableaux.

```

In [45]: def gru_backward(dh, caches):
    """
    Calcule les gradients pour un GRU sur une séquence de T_x pas de temps.

    Paramètres :
        dh      -- Gradients des états cachés, forme (d_h, n, T_x)
        caches  -- Liste des caches de chaque cellule GRU

    Retour :
        gradients -- Dictionnaire contenant :
            dx      : Gradient des entrées, forme (d_x, n, T_x)
            dh0     : Gradient de l'état initial h0, forme (d_h, n)
            dWu, dWr, dWh : Gradients des matrices de poids
            dbu, dbr, dbh : Gradients des biais
    """
    caches_list, x = caches
    # ... = x.shape
    # ... = dh.shape[0]
    ##### 2 lignes #####
    d_x, n, T_x = x.shape
    d_h = dh.shape[0]
    #####

    ## Initialisation des gradients à zéro
    # dx, dWu, dWr, dWh, dbu, dbr, dbh, dh_prev = ...
    ##### 1~8 lignes #####
    dx = np.zeros((d_x, n, T_x))
    dWu = np.zeros_like(caches_list[0][-1]['Wu'])
    dWr = np.zeros_like(caches_list[0][-1]['Wr'])
    dWh = np.zeros_like(caches_list[0][-1]['Wh'])
    dbu = np.zeros((d_h, 1))
    dbr = np.zeros((d_h, 1))
    dbh = np.zeros((d_h, 1))
    dh_prev = np.zeros((d_h, n)) # Gradient accumulé de h_{t-1}
    #####

    ## Boucle inverse sur les pas de temps
    for t in reversed(range(T_x)):
        gradients = gru_cell_backward(dh[:, :, t] + dh_prev, caches_list[t])
        ##### 8 lignes #####
        dx[:, :, t] = gradients["dxt"]
        dWu += gradients["dWu"]
        dWr += gradients["dWr"]
        dWh += gradients["dWh"]
        dbu += gradients["dbu"]
        dbr += gradients["dbr"]
        dbh += gradients["dbh"]
        dh_prev = gradients["dh_prev"] # Propagation du gradient à t-1
        #####

    dh0 = dh_prev # Gradient final de l'état initial h0

    gradients = {
        "dx": dx, "dh0": dh0,
        "dWu": dWu, "dWr": dWr, "dWh": dWh,
        "dbu": dbu, "dbr": dbr, "dbh": dbh
    }
    return gradients

```

```
In [47]: # Test corrigé de la cellule GRU
np.random.seed(1)
xt = np.random.randn(3, 10)
h_prev = np.random.randn(5, 10)

# Ajouter uy et by dans les paramètres
uy = np.random.randn(2, 5) # Supposons d_y = 2
by = np.random.randn(2, 1)
parameters = {
    "Wu": np.random.randn(5, 8),
    "Wr": np.random.randn(5, 8),
    "Wh": np.random.randn(5, 8),
    "bu": np.zeros((5, 1)),
    "br": np.zeros((5, 1)),
    "bh": np.zeros((5, 1)),
    "uy": uy, # <-- Clé ajoutée
    "by": by  # <-- Clé ajoutée
}

h_next, yt, cache = gru_cell_forward(xt, h_prev, parameters)
print("h_next.shape =", h_next.shape)

h_next.shape = (5, 10)
```

Output attendus:

```
h_next.shape = (5, 10)
```

2 - Comparaison entre RNN, LSTM et GRU pour le traitement de la langue française (à lire)

Dans cette section, nous allons comparer les performances des réseaux de neurones récurrents traditionnels (RNN), des réseaux de mémoire à long terme (LSTM) et des réseaux à unités récurrentes à portes (GRU) sur une tâche spécifique de traitement de la langue française. Nous utiliserons un exemple concret de prédiction de la prochaine lettre dans une séquence de mots français.

- Description de la tâche

La tâche consiste à prédire la prochaine lettre dans une séquence de mots français. Par exemple, si l'entrée est "bonjou", le modèle doit prédire que la prochaine lettre est "r".

- Préparation des données

Nous allons utiliser un corpus de mots français pour entraîner nos modèles. Les mots seront tokenisés en caractères, et nous allons créer des séquences d'entrée et de sortie correspondantes.

```
In [2]: # Exemple de corpus de mots français
corpus = ["bonjour", "merci", "salut", "au revoir", "s'il vous plaît"]

# Tokenisation des mots en caractères
chars = sorted(list(set("".join(corpus))))
char_to_idx = {char: idx for idx, char in enumerate(chars)}
idx_to_char = {idx: char for idx, char in enumerate(chars)}

# Création des séquences d'entrée et de sortie
def prepare_sequences(corpus, char_to_idx, seq_length):
    X = []
    y = []
    for word in corpus:
        for i in range(len(word) - seq_length):
            seq_in = word[i:i + seq_length]
            seq_out = word[i + seq_length]
            X.append([char_to_idx[char] for char in seq_in])
            y.append(char_to_idx[seq_out])
    return np.array(X), np.array(y)

seq_length = 5
X, y = prepare_sequences(corpus, char_to_idx, seq_length)
print("X.shape =", X.shape)
print("y.shape =", y.shape)

X.shape = (16, 5)
y.shape = (16,)
```

Output attendus:

X.shape = (16, 5)

y.shape = (16,)

- Implémentation des modèles

Nous allons maintenant implémenter les trois modèles : RNN, LSTM et GRU.

2.1 - RNN traditionnel

- **Initialisation des paramètres** : Les poids (w , v , u) et les biais b sont initialisés avec de petites valeurs aléatoires pour éviter des problèmes de saturation des gradients.
- **Propagation avant** : La méthode `rnn_forward` calcule les états cachés et les prédictions pour chaque pas de temps dans la séquence.
- **Calcul de la perte** : La perte est calculée en utilisant l'entropie croisée ($L = -\frac{1}{n} \sum_{i=1}^n y_i \ln(\hat{y}_i)$), qui est une mesure courante pour les problèmes de classification.
- **Rétropropagation** : La méthode `rnn_backward` calcule les gradients des paramètres par rapport à la perte et les utilise pour mettre à jour les poids et les biais.
- **Entraînement** : La méthode `train` itère sur les époques, effectuant la propagation avant, le calcul de la perte, la rétropropagation et la mise à jour des paramètres.
- **Prédiction** : La fonction `predict_next_char` utilise le modèle entraîné pour prédire le caractère suivant dans une séquence donnée.

Exercice 5 Réalisez un réseau RNN complet.

Instructions: Reportez-vous aux commentaires dans Td5 et à la section code.


```

In [3]: ## Réseau RNN
# Toutes les fonctions sont organisées dans une classe complète
# Note : Le calcul de perte spécifique au RNN doit encore être implémenté
# La structure "neurone vers réseau" n'est plus utilisée pour la rétropropagation classique

class RNN: simple
    ## Définissez les dimensions d'entrée (d_x), cachée (d_h) et de sortie (d_y)
    def __init__(self, vocab_size, hidden_dim):
        ## Taille du vocabulaire, dimension de l'espace caché, taille de la sortie: (d_x, d_h, d_y)
        ## Initialisation des paramètres
        # self.d_x = ...
        # self.d_h = ...
        # self.d_y = ...
        # self.parameters = ...
        ##### 4 lignes #####
        self.d_x = vocab_size
        self.d_h = hidden_dim
        self.d_y = vocab_size
        self.parameters = self._init_parameters()
        #####

    ## Initialisez les matrices de poids (w, v, u) avec de petites valeurs aléatoires
    ## Initialisez les biais (bh, by) à zéro
    def _init_parameters(self):
        np.random.seed(1)
        ## Initialisation des poids et biais
        # Poids pour l'entrée : 'w': np.random.randn(...) * 0.01
        # Poids pour l'état caché : 'v': np.random.randn(...) * 0.01
        # Poids pour la sortie : 'u': np.random.randn(...) * 0.01
        # Biais pour l'état caché : 'bh': np.zeros((...))
        # Biais pour la sortie : 'by': np.zeros((...))
        return {
            ##### 5 lignes #####
            'w': np.random.randn(self.d_x, self.d_h) * 0.01,
            'v': np.random.randn(self.d_h, self.d_h) * 0.01,
            'u': np.random.randn(self.d_h, self.d_y) * 0.01,
            'bh': np.zeros((self.d_h, 1)),
            'by': np.zeros((self.d_y, 1))
            #####
        }

    def rnn_cell_forward(self, xt, h_prev):
        ## Récupération des paramètres
        # W, V, U = ...
        # bh, by = ...
        ##### 2~5 lignes #####
        W, V, U = self.parameters['w'], self.parameters['v'], self.parameters['u']
        bh, by = self.parameters['bh'], self.parameters['by']
        #####
        ## Calcul de l'état caché suivant
        # h_next = ...
        ##### 1 ligne #####
        h_next = np.tanh(W.T @ xt + V.T @ h_prev + bh)
        #####
        ## Calcul de la prédiction de la sortie
        # yt_pred = ...
        ##### 1 ligne #####
        yt_pred = softmax(U.T @ h_next + by)
        #####
        return h_next, yt_pred, (h_next, h_prev, xt)

    def rnn_forward(self, X):
        n, T_x = X.shape[1], X.shape[2]
        ## Initialisation des états cachés et des prédictions
        # h, y_pred, caches, h_prev = ...
        ##### 1~4 lignes #####
        h = np.zeros((self.d_h, n, T_x))
        y_pred = np.zeros((self.d_y, n, T_x))
        caches = []
        h_prev = np.zeros((self.d_h, n))
        #####
        ## Propagation avant à travers le temps
        # for-loop
        ##### ~5 lignes #####
        for t in range(T_x):
            h_prev, yt, cache = self.rnn_cell_forward(X[:, :, t], h_prev)
            h[:, :, t] = h_prev
            y_pred[:, :, t] = yt
            caches.append(cache)
        #####
        return h, y_pred, caches

```

```

def compute_loss(self, y_pred, y_true):
    ## Calcul de la perte sur la dernière étape
    ##### 2~3 lignes #####
    last_step_pred = y_pred[:, :, -1]
    return -np.mean(np.sum(y_true * np.log(last_step_pred + 1e-8), axis=0))
    #####

def rnn_backward(self, X, y_true, caches, y_pred):
    n = X.shape[1]
    T_x = X.shape[2]
    ## Initialisation des gradients
    # grads = ...
    # dh_next = ...
    ##### 2 lignes #####
    grads = {k: np.zeros_like(v) for k, v in self.parameters.items()}
    dh_next = np.zeros((self.d_h, n))
    #####

    ## Calcul du gradient pour la dernière étape
    # dy = ...
    # dy[:, :, -1] = ...
    ##### 2 lignes #####
    dy = np.zeros((self.d_y, n, T_x))
    dy[:, :, -1] = (y_pred[:, :, -1] - y_true) / n
    #####

    ## Rétropropagation à travers le temps
    # for-loop
    for t in reversed(range(T_x)):
        h_next, h_prev, xt = caches[t]
        ## Gradient pour la sortie
        # dU = ...
        # dby = ...
        ##### 2 lignes #####
        dU = h_next @ dy[:, :, t].T
        dby = np.sum(dy[:, :, t], axis=1, keepdims=True)
        #####

        ## Gradient pour l'état caché
        # dh = ...
        # dtanh = ...
        ##### 2 lignes #####
        dh = self.parameters['u'] @ dy[:, :, t] + dh_next
        dtanh = (1 - h_next**2) * dh
        #####

        ## Mise à jour des gradients des paramètres
        # grads['w'] += ...
        # grads['v'] += ...
        # grads['bh'] += ...
        ##### 3 lignes #####
        grads['w'] += xt @ dtanh.T
        grads['v'] += h_prev @ dtanh.T
        grads['bh'] += np.sum(dtanh, axis=1, keepdims=True)
        #####

        ## Propagation du gradient à l'étape précédente
        # dh_next = ...
        ##### 1 ligne #####
        dh_next = self.parameters['v'] @ dtanh
        #####
    return grads

def train(self, X, y_true, lr=0.1, epochs=100):
    # Conversion des étiquettes en encodage one-hot
    y_true_onehot = np.zeros((self.d_y, X.shape[1]))
    y_true_onehot[y_true] = 1
    # Boucle d'entraînement
    for epoch in range(epochs):
        # Propagation avant
        _, y_pred, caches = self.rnn_forward(X)
        # Calcul de la perte
        loss = self.compute_loss(y_pred, y_true_onehot)
        # Rétropropagation
        grads = self.rnn_backward(X, y_true_onehot, caches, y_pred)
        # Mise à jour des paramètres
        for k in self.parameters:
            self.parameters[k] -= lr * grads[k]
        # Affichage de la perte
        if epoch % 100 == 0:
            print(f"Epoch {epoch} Loss: {loss:.4f}")

```

```

In [12]: # Test du code
def create_dataset(corpus, seq_length):
    X, y = [], []
    for word in corpus:
        for i in range(len(word)-seq_length):
            seq_in = word[i:i+seq_length]
            seq_out = word[i+seq_length]
            X.append([char_to_idx[c] for c in seq_in])
            y.append(char_to_idx[seq_out])

    # Conversion en encodage one-hot
    vocab_size = len(chars)
    X_onehot = np.zeros((len(X), seq_length, vocab_size))
    for i, seq in enumerate(X):
        for t, idx in enumerate(seq):
            X_onehot[i, t, idx] = 1
    return X_onehot.transpose(2, 0, 1), np.array(y) # (d_x, n, T_x)

# Hyperparamètres
SEQ_LENGTH = 10
HIDDEN_DIM = 20
EPOCHS = 1000

# Préparation des données
X, y = create_dataset(corpus, SEQ_LENGTH)

model = RNN(vocab_size=len(chars), hidden_dim=HIDDEN_DIM)

# Entraînement du modèle
model.train(X, y, lr=0.2, epochs=EPOCHS)

# Prédiction d'un caractère suivant
def predict_next_char(model, seed_seq):
    seed_idx = [char_to_idx[c] for c in seed_seq]
    x = np.zeros((len(chars), 1, len(seed_seq)))
    for t, idx in enumerate(seed_idx):
        x[idx, 0, t] = 1
    _, y_pred, _ = model.rnn_forward(x)
    return idx_to_char[np.argmax(y_pred[:, 0, -1])]

# Test de prédiction
test_seq = "bonjou"
print(f"Predicted next character for '{test_seq}': {predict_next_char(model, test_seq)}")

Epoch 0 Loss: 14.7224
Epoch 100 Loss: 14.4674
Epoch 200 Loss: 14.4541
Epoch 300 Loss: 14.4516
Epoch 400 Loss: 14.4505
Epoch 500 Loss: 14.4499
Epoch 600 Loss: 14.4495
Epoch 700 Loss: 14.4493
Epoch 800 Loss: 14.4491
Epoch 900 Loss: 14.4489
Predicted next character for 'bonjou': l

```

2.2 - LSTM

- **Structure LSTM :**
 - Le LSTM comporte trois portes principales (oubli, entrée, sortie) et un état mémoire
 - Chaque porte a ses propres paramètres (W, b) qui sont appris pendant l'entraînement
- **Flux de données :**
 - Propagation avant : Calcul séquentiel des états cachés/mémoire et des prédictions
 - Rétropropagation : Calcul précis des gradients à travers les différentes portes
- **Optimisations :**
 - Initialisation soignée des paramètres (valeurs petites et aléatoires)
 - Ajout d'un terme epsilon (1e-8) pour éviter les problèmes numériques
 - Stockage efficace des valeurs intermédiaires pour la rétropropagation
- **Entraînement :**
 - Descentes de gradient avec taux d'apprentissage configurable
 - Affichage périodique de la perte pour suivre la convergence
- **Particularités :**
 - Mécanisme de mémoire à long terme (état c)
 - Portes apprenant quoi oublier/conservé (combinaison non linéaire)

Exercice 6 Réalisez un réseau LSTM complet.

Instructions: Reportez-vous aux commentaires dans Td5 et à la section code.


```

In [7]: ## Réseau LSTM (Long Short-Term Memory)
# Toutes les fonctions sont organisées dans une classe complète
# Note : Le calcul de perte spécifique au LSTM doit encore être implémenté
# La structure "neurone vers réseau" n'est plus utilisée pour la rétropropagation classique
class LSTM:
    def __init__(self, input_dim, hidden_dim, output_dim):
        self.d_x = input_dim # Dimension de l'entrée
        self.d_h = hidden_dim # Dimension des états cachés
        self.d_y = output_dim # Dimension de sortie
        self.parameters = self._initialize_parameters() # Initialisation des paramètres

    def _initialize_parameters(self):
        """Initialisation des poids et biais du LSTM"""
        np.random.seed(1) # Pour la reproductibilité
        d = self.d_x + self.d_h # Dimension combinée pour les portes

        # Paramètres des portes:
        Wf = np.random.randn(self.d_h, d) * 0.01 # Porte d'oubli (Forget)
        Wi = np.random.randn(self.d_h, d) * 0.01 # Porte d'entrée (Input)
        Wc = np.random.randn(self.d_h, d) * 0.01 # Porte de mémoire (Cell state)
        Wo = np.random.randn(self.d_h, d) * 0.01 # Porte de sortie (Output)

        # Couche de sortie:
        Wy = np.random.randn(self.d_y, self.d_h) * 0.01 # Poids de sortie

        # Biais:
        bf = np.zeros((self.d_h, 1)) # Biais porte d'oubli
        bi = np.zeros((self.d_h, 1)) # Biais porte d'entrée
        bc = np.zeros((self.d_h, 1)) # Biais état mémoire
        bo = np.zeros((self.d_h, 1)) # Biais porte de sortie
        by = np.zeros((self.d_y, 1)) # Biais sortie finale

        return {
            "Wf": Wf, "Wi": Wi, "Wc": Wc, "Wo": Wo, # Poids des portes
            "Wy": Wy, # Poids de sortie
            "bf": bf, "bi": bi, "bc": bc, "bo": bo, # Biais des portes
            "by": by # Biais de sortie
        }

    def lstm_cell_forward(self, xt, h_prev, c_prev):
        """Propagation avant pour une seule cellule LSTM"""
        # Récupération des paramètres
        Wf, bf = self.parameters["Wf"], self.parameters["bf"]
        Wi, bi = self.parameters["Wi"], self.parameters["bi"]
        Wc, bc = self.parameters["Wc"], self.parameters["bc"]
        Wo, bo = self.parameters["Wo"], self.parameters["bo"]
        Wy, by = self.parameters["Wy"], self.parameters["by"]

        # Concaténation de l'entrée et de l'état caché précédent
        concat = np.vstack((xt, h_prev))

        # Calcul des portes:
        ft = sigmoid(np.dot(Wf, concat) + bf) # Porte d'oubli
        it = sigmoid(np.dot(Wi, concat) + bi) # Porte d'entrée
        cct = np.tanh(np.dot(Wc, concat) + bc) # Candidat pour l'état mémoire
        ot = sigmoid(np.dot(Wo, concat) + bo) # Porte de sortie

        # Mise à jour de l'état mémoire:
        c_next = ft * c_prev + it * cct

        # Calcul du nouvel état caché:
        h_next = ot * np.tanh(c_next)

        # Calcul de la prédiction:
        yt_pred = softmax(np.dot(Wy, h_next) + by)

        # Stockage pour la rétropropagation (9 éléments nécessaires)
        cache = (
            h_next.copy(), # 0 - Nouvel état caché
            c_next.copy(), # 1 - Nouvel état mémoire
            h_prev.copy(), # 2 - État caché précédent
            c_prev.copy(), # 3 - État mémoire précédent
            ft.copy(), # 4 - Porte d'oubli
            it.copy(), # 5 - Porte d'entrée
            cct.copy(), # 6 - Candidat mémoire
            ot.copy(), # 7 - Porte de sortie
            xt.copy() # 8 - Entrée actuelle
        )

        return h_next, c_next, yt_pred, cache

```

```

def lstm_forward(self, x, h0):
    """Propagation avant complète pour toutes les séquences"""
    d_x, n, T_x = x.shape # Dimensions: entrée × échantillons × pas de temps
    h = np.zeros((self.d_h, n, T_x)) # États cachés
    c = np.zeros((self.d_h, n, T_x)) # États mémoire
    y = np.zeros((self.d_y, n, T_x)) # Prédictions
    caches = [] # Pour stocker toutes les informations

    # Initialisation des états
    h_prev = h0
    c_prev = np.zeros_like(h0)

    for t in range(T_x):
        # Propagation au pas de temps t
        h_next, c_next, yt, cache = self.lstm_cell_forward(
            x[:, :, t], h_prev, c_prev
        )

        # Stockage des résultats
        h[:, :, t] = h_next
        c[:, :, t] = c_next
        y[:, :, t] = yt
        caches.append(cache)

        # Mise à jour pour l'itération suivante
        h_prev = h_next
        c_prev = c_next

    return h, c, y, (caches, x)

def lstm_cell_backward(self, dh_next, dc_next, cache):
    """Rétropropagation pour une seule cellule LSTM"""
    # Récupération des valeurs stockées
    h_next, c_next, h_prev, c_prev, ft, it, cct, ot, xt = cache
    d_x, n = xt.shape

    # Calcul des gradients pour les portes:
    dot = dh_next * np.tanh(c_next) * ot * (1 - ot)
    dcct = (dh_next * ot * (1 - np.tanh(c_next)**2) + dc_next) * it * (1 - cct**2)
    dit = (dh_next * ot * (1 - np.tanh(c_next)**2) + dc_next) * cct * it * (1 - it)
    dft = (dh_next * ot * (1 - np.tanh(c_next)**2) + dc_next) * c_prev * ft * (1 - ft)

    # Gradients des paramètres:
    concat = np.vstack((xt, h_prev)).T
    dWf = np.dot(dft, concat)
    dWi = np.dot(dit, concat)
    dWc = np.dot(dcct, concat)
    dWo = np.dot(dot, concat)

    dbf = np.sum(dft, axis=1, keepdims=True)
    dbi = np.sum(dit, axis=1, keepdims=True)
    dbc = np.sum(dcct, axis=1, keepdims=True)
    dbo = np.sum(dot, axis=1, keepdims=True)

    # Gradients pour l'étape précédente:
    dxt = np.dot(self.parameters["Wf"][:, :d_x].T, dft) + \
        np.dot(self.parameters["Wi"][:, :d_x].T, dit) + \
        np.dot(self.parameters["Wc"][:, :d_x].T, dcct) + \
        np.dot(self.parameters["Wo"][:, :d_x].T, dot)

    dh_prev = np.dot(self.parameters["Wf"][:, d_x:].T, dft) + \
        np.dot(self.parameters["Wi"][:, d_x:].T, dit) + \
        np.dot(self.parameters["Wc"][:, d_x:].T, dcct) + \
        np.dot(self.parameters["Wo"][:, d_x:].T, dot)

    dc_prev = (dh_next * ot * (1 - np.tanh(c_next)**2) + dc_next) * ft

    # Organisation des gradients dans un dictionnaire
    gradients = {
        "dxt": dxt, "dh_prev": dh_prev, "dc_prev": dc_prev,
        "dWf": dWf, "dbf": dbf, "dWi": dWi, "dbi": dbi,
        "dWc": dWc, "dbc": dbc, "dWo": dWo, "dbo": dbo
    }

    return gradients

def lstm_backward(self, dh, dc, caches):
    """Rétropropagation complète à travers le temps"""
    (caches, x) = caches
    _, _, h_prev, c_prev, _, _, _, _, xt = caches[0]

    # Initialisation des gradients

```

```

dx = np.zeros_like(x)
dh0 = np.zeros_like(h_prev)
dc0 = np.zeros_like(c_prev)

gradients = {
    'dwf': np.zeros_like(self.parameters['Wf']),
    'dwi': np.zeros_like(self.parameters['Wi']),
    'dwc': np.zeros_like(self.parameters['Wc']),
    'dwo': np.zeros_like(self.parameters['Wo']),
    'dbf': np.zeros_like(self.parameters['bf']),
    'dbi': np.zeros_like(self.parameters['bi']),
    'dbc': np.zeros_like(self.parameters['bc']),
    'dbo': np.zeros_like(self.parameters['bo'])
}

# Initialisation pour la rétropropagation
dh_next = np.zeros_like(h_prev)
dc_next = np.zeros_like(c_prev)

# Parcours inverse dans le temps
for t in reversed(range(dh.shape[2])):
    # Récupération du cache actuel
    cache = caches[t]

    # Calcul des gradients
    grads = self.lstm_cell_backward(
        dh[:, :, t] + dh_next,
        dc_next,
        cache
    )

    # Mise à jour des gradients
    dx[:, :, t] = grads['dxt']
    dh_next = grads['dh_prev']
    dc_next = grads['dc_prev']

    for param in gradients:
        gradients[param] += grads[param]

gradients['dh0'] = dh_next
gradients['dc0'] = dc_next
gradients['dx'] = dx

return gradients

def compute_loss(self, y_pred, y_true):
    """Calcul de la perte d'entropie croisée"""
    m = y_true.shape[1] # Nombre d'échantillons
    return -np.sum(y_true * np.log(y_pred + 1e-8)) / m # +1e-8 pour éviter log(0)

def train(self, X, Y, learning_rate=0.01, epochs=100):
    """Entraînement du modèle LSTM"""
    n_samples = X.shape[1]
    h0 = np.zeros((self.d_h, n_samples)) # Initialisation de l'état caché

    for epoch in range(epochs):
        # Propagation avant
        h, c, y_pred, caches = self.lstm_forward(X, h0)

        # Calcul de la perte
        loss = self.compute_loss(y_pred[:, :, -1], Y)

        # Initialisation des gradients
        dh = np.zeros_like(h)
        dc = np.zeros_like(c)

        # Gradient au dernier pas de temps
        dh[:, :, -1] = np.dot(self.parameters['Wy'].T, (y_pred[:, :, -1] - Y))

        # Rétropropagation
        gradients = self.lstm_backward(dh, dc, caches)

        # Mise à jour des paramètres
        for param in gradients:
            if param.startswith('dw') or param.startswith('db'):
                self.parameters[param[1:]] -= learning_rate * gradients[param]

        # Affichage périodique
        if epoch % 10 == 0:
            print(f"Epoch {epoch} Perte: {loss:.4f}")

```



```

In [8]: # Test du code
def create_dataset(corpus, seq_length=3):
    X, Y = [], []
    for word in corpus:
        for i in range(len(word)-seq_length):
            seq = [char_to_idx[c] for c in word[i:i+seq_length]]
            target = char_to_idx[word[i+seq_length]]
            X.append(seq)
            Y.append(target)

    vocab_size = len(chars)
    X_onehot = np.zeros((len(X), vocab_size, seq_length))
    for i, seq in enumerate(X):
        for t, idx in enumerate(seq):
            X_onehot[i, idx, t] = 1

    return X_onehot.transpose(1, 0, 2), np.eye(vocab_size)[Y].T

# Hyperparamètres
SEQ_LENGTH = 10
HIDDEN_DIM = 20
EPOCHS = 100

X, Y = create_dataset(corpus)

# Initialisation
model = LSTM(input_dim=len(chars), hidden_dim=HIDDEN_DIM, output_dim=len(chars))

# Entraînement
model.train(X, Y, learning_rate=0.01, epochs=EPOCHS)

# Prediction
def predict_next_char(model, seed_seq):
    seed_idx = [char_to_idx[c] for c in seed_seq]
    x = np.zeros((len(chars), 1, len(seed_seq)))
    for t, idx in enumerate(seed_idx):
        x[idx, 0, t] = 1
    _, _, y_pred, _ = model.lstm_forward(x, np.zeros((32, 1)))
    return chars[np.argmax(y_pred[:, 0, -1])]

print(predict_next_char(model, "bonjou")) # 应输出 'r'

Epoch 0 Perte: 2.9444
Epoch 10 Perte: 2.9444
Epoch 20 Perte: 2.9443
Epoch 30 Perte: 2.9443
Epoch 40 Perte: 2.9443
Epoch 50 Perte: 2.9442
Epoch 60 Perte: 2.9442
Epoch 70 Perte: 2.9442
Epoch 80 Perte: 2.9441
Epoch 90 Perte: 2.9441

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In [8], line 41
    38     _, _, y_pred, _ = model.lstm_forward(x, np.zeros((32, 1)))
    39     return chars[np.argmax(y_pred[:, 0, -1])]
--> 41 print(predict_next_char(model, "bonjour")) # 应输出 'r'

Cell In [8], line 38, in predict_next_char(model, seed_seq)
    36 for t, idx in enumerate(seed_idx):
    37     x[idx, 0, t] = 1
--> 38     _, _, y_pred, _ = model.lstm_forward(x, np.zeros((32, 1)))
    39     return chars[np.argmax(y_pred[:, 0, -1])]

Cell In [7], line 96, in LSTM.lstm_forward(self, x, h0)
    92 c_prev = np.zeros_like(h0)
    94 for t in range(T_x):
    95     # Propagation au pas de temps t
--> 96     h_next, c_next, yt, cache = self.lstm_cell_forward(
    97         x[:, :, t], h_prev, c_prev
    98     )
    99     # Stockage des résultats
   100     h[:, :, t] = h_next
   101     h[:, :, t] = h_next

Cell In [7], line 53, in LSTM.lstm_cell_forward(self, xt, h_prev, c_prev)
    50 concat = np.vstack((xt, h_prev))
    52 # Calcul des portes:
--> 53 ft = sigmoid(np.dot(Wf, concat) + bf) # Porte d'oubli
    54 it = sigmoid(np.dot(Wi, concat) + bi) # Porte d'entrée
    55 cct = np.tanh(np.dot(Wc, concat) + bc) # Candidat pour l'état mémoire

File <__array_function__ internals>:200, in dot(*args, **kwargs)

ValueError: shapes (20,39) and (51,1) not aligned: 39 (dim 1) != 51 (dim 0)

```

2.3 - GRU

- **Initialisation des paramètres** : Les poids (w, v, u) et les biais b sont initialisés avec de petites valeurs aléatoires pour éviter des problèmes de saturation des gradients.
- **Propagation avant** : La méthode `rnn_forward` calcule les états cachés et les prédictions pour chaque pas de temps dans la séquence.
- **Calcul de la perte** : La perte est calculée en utilisant l'entropie croisée ($L = -\frac{1}{n} \sum_{i=1}^n y_i \ln(\hat{y}_i)$), qui est une mesure courante pour les problèmes de classification.
- **Rétropropagation** : La méthode `rnn_backward` calcule les gradients des paramètres par rapport à la perte et les utilise pour mettre à jour les poids et les biais.
- **Entraînement** : La méthode `train` itère sur les époques, effectuant la propagation avant, le calcul de la perte, la rétropropagation et la mise à jour des paramètres.
- **Prédiction** : La fonction `predict_next_char` utilise le modèle entraîné pour prédire le caractère suivant dans une séquence donnée.

Exercice 7 Réalisez un réseau GRU complet.

Instructions: Reportez-vous aux commentaires dans Td6 et à la section code.


```

In [12]: ## Réseau GRU (Gated Recurrent Unit)
# Toutes les fonctions sont organisées dans une classe complète
# Note : Le calcul de perte spécifique au LSTM doit encore être implémenté
# La structure "neurone vers réseau" n'est plus utilisée pour la rétropropagation classique

class GRU:
    def __init__(self, input_dim, hidden_dim, output_dim):
        """
        Initialise un réseau GRU (Gated Recurrent Unit).

        Paramètres :
        input_dim -- Dimension d'entrée (d_x)
        hidden_dim -- Dimension de l'état caché (d_h)
        output_dim -- Dimension de sortie (d_y)
        """
        self.d_x = input_dim
        self.d_h = hidden_dim
        self.d_y = output_dim
        self.parameters = self._initialize_parameters()

    def _initialize_parameters(self):
        """Initialisation des paramètres du GRU"""
        np.random.seed(1) # Pour la reproductibilité
        d = self.d_x + self.d_h # Dimension combinée

        # Portes de mise à jour et réinitialisation
        Wu = np.random.randn(self.d_h, d) * 0.01 # Porte de mise à jour
        Wr = np.random.randn(self.d_h, d) * 0.01 # Porte de réinitialisation
        Wh = np.random.randn(self.d_h, d) * 0.01 # Candidat état caché

        # Couche de sortie
        Wy = np.random.randn(self.d_y, self.d_h) * 0.01

        # Biais
        bu = np.zeros((self.d_h, 1)) # Porte de mise à jour
        br = np.zeros((self.d_h, 1)) # Porte de réinitialisation
        bh = np.zeros((self.d_h, 1)) # Candidat état caché
        by = np.zeros((self.d_y, 1)) # Sortie

        return {
            "Wu": Wu, "Wr": Wr, "Wh": Wh, "Wy": Wy,
            "bu": bu, "br": br, "bh": bh, "by": by
        }

    def gru_cell_forward(self, xt, h_prev):
        """
        Propagation avant pour une cellule GRU unique.

        Paramètres :
        xt -- Entrée courante (d_x, n)
        h_prev -- État caché précédent (d_h, n)

        Retourne :
        h_next -- Nouvel état caché
        yt_pred -- Prédiction courante
        cache -- Valeurs intermédiaires pour la rétropropagation
        """
        # Récupération des paramètres
        Wu, Wr, Wh = self.parameters['Wu'], self.parameters['Wr'], self.parameters['Wh']
        bu, br, bh = self.parameters['bu'], self.parameters['br'], self.parameters['bh']
        Wy, by = self.parameters['Wy'], self.parameters['by']

        # Concaténation entrée + état caché précédent
        concat = np.vstack((xt, h_prev)) # (d_x + d_h, n)

        # Calcul des portes
        gamma_u = sigmoid(np.dot(Wu, concat) + bu) # Porte de mise à jour
        gamma_r = sigmoid(np.dot(Wr, concat) + br) # Porte de réinitialisation

        # État caché candidat
        h_tilde = np.tanh(np.dot(Wh, np.vstack((gamma_r * h_prev, xt))) + bh)

        # Nouvel état caché (combinaison)
        h_next = gamma_u * h_prev + (1 - gamma_u) * h_tilde

        # Prédiction de sortie
        yt_pred = softmax(np.dot(Wy, h_next) + by)

        # Stockage pour la rétropropagation
        cache = (h_next, h_prev, gamma_u, gamma_r, h_tilde, xt)

```

```

        return h_next, yt_pred, cache

def gru_forward(self, x, h0):
    """
    Propagation avant complète pour le GRU.

    Paramètres :
    x -- Séquence d'entrée (d_x, n, T_x)
    h0 -- État caché initial (d_h, n)

    Retourne :
    h -- États cachés (d_h, n, T_x)
    y -- Prédictions (d_y, n, T_x)
    caches -- Valeurs intermédiaires pour chaque pas de temps
    """
    d_x, n, T_x = x.shape
    h = np.zeros((self.d_h, n, T_x))
    y = np.zeros((self.d_y, n, T_x))
    h_prev = h0
    caches = []

    for t in range(T_x):
        # Propagation au pas de temps t
        h_next, yt, cache = self.gru_cell_forward(x[:, :, t], h_prev)

        # Stockage des résultats
        h[:, :, t] = h_next
        y[:, :, t] = yt
        caches.append(cache)

        # Mise à jour pour l'itération suivante
        h_prev = h_next

    return h, y, (caches, x)

def gru_cell_backward(self, dh_next, cache):
    """
    Rétropropagation pour une cellule GRU unique.

    Paramètres :
    dh_next -- Gradient du prochain état caché (d_h, n)
    cache -- Valeurs stockées pendant la propagation avant

    Retourne :
    gradients -- Dictionnaire contenant tous les gradients
    """
    h_next, h_prev, gamma_u, gamma_r, h_tilde, xt = cache
    n = xt.shape[1] # Taille du batch
    d_h = h_next.shape[0]
    d_x = xt.shape[0]

    # Gradients locaux
    dsigma_u = dh_next * (h_prev - h_tilde) * gamma_u * (1 - gamma_u)
    dtanh = dh_next * (1 - gamma_u) * (1 - h_tilde ** 2)

    # Gradient pour la porte de réinitialisation
    Wh_h = self.parameters['Wh'][:, :d_h] # Partie concernant h_prev
    dsigma_r = np.dot(Wh_h.T, dtanh) * h_prev * gamma_r * (1 - gamma_r)

    # Concaténation pour les portes
    concat = np.vstack((xt, h_prev))

    # Gradients des poids
    dWu = np.dot(dsigma_u, concat.T)
    dWr = np.dot(dsigma_r, concat.T)
    dWh = np.dot(dtanh, np.vstack((gamma_r * h_prev, xt)).T)

    # Gradients des biais
    dbu = np.sum(dsigma_u, axis=1, keepdims=True)
    dbr = np.sum(dsigma_r, axis=1, keepdims=True)
    dbh = np.sum(dtanh, axis=1, keepdims=True)

    # Gradients pour l'entrée et l'état précédent
    dxt = (
        np.dot(self.parameters['Wu'][:, :d_x].T, dsigma_u) +
        np.dot(self.parameters['Wr'][:, :d_x].T, dsigma_r) +
        np.dot(self.parameters['Wh'][:, :d_x].T, dtanh)
    )

    dh_prev = (
        dh_next * gamma_u + # Contribution directe
        np.dot(self.parameters['Wu'][:, d_x:].T, dsigma_u) +

```

```

        np.dot(self.parameters['Wr'][:,d_x:].T, dsigma_r) +
        gamma_r * np.dot(Wh_h.T, dtanh) # Contribution via la porte de réinitialisation
    )

    gradients = {
        "dxt": dxt, "dh_prev": dh_prev,
        "dWu": dWu, "dWr": dWr, "dWh": dWh,
        "dbu": dbu, "dbr": dbr, "dbh": dbh
    }

    return gradients

def gru_backward(self, dh, caches):
    """
    Rétropropagation dans le temps pour le GRU.

    Paramètres :
    dh -- Gradients des états cachés (d_h, n, T_x)
    caches -- Valeurs stockées pendant la propagation avant

    Retourne :
    gradients -- Dictionnaire contenant tous les gradients
    """
    (caches, x) = caches
    d_x, n, T_x = x.shape
    d_h = dh.shape[0]

    # Initialisation des gradients
    dx = np.zeros_like(x)
    gradients = {
        'dWu': np.zeros_like(self.parameters['Wu']),
        'dWr': np.zeros_like(self.parameters['Wr']),
        'dWh': np.zeros_like(self.parameters['Wh']),
        'dbu': np.zeros_like(self.parameters['bu']),
        'dbr': np.zeros_like(self.parameters['br']),
        'dbh': np.zeros_like(self.parameters['bh'])
    }
    dh_prev = np.zeros((d_h, n))

    # Parcours inverse dans le temps
    for t in reversed(range(T_x)):
        # Calcul des gradients
        grads = self.gru_cell_backward(dh[:, :, t] + dh_prev, caches[t])

        # Mise à jour des gradients
        dx[:, :, t] = grads["dxt"]
        dh_prev = grads["dh_prev"]

        for param in gradients:
            gradients[param] += grads[param]

    gradients["dx"] = dx
    gradients["dh0"] = dh_prev

    return gradients

def compute_loss(self, y_pred, y_true):
    """
    Calcule la perte d'entropie croisée.

    Paramètres :
    y_pred -- Prédications (d_y, n, T_x)
    y_true -- Vraies valeurs (d_y, n, T_x)

    Retourne :
    loss -- Valeur de la perte
    """
    m = y_true.shape[1] # Nombre d'échantillons
    # Éviter log(0) avec un petit epsilon
    loss = -np.sum(y_true * np.log(y_pred + 1e-8)) / m
    return loss

def train(self, X, Y, learning_rate=0.01, epochs=100):
    """
    Entraîne le modèle GRU.

    Paramètres :
    X -- Données d'entrée (d_x, n, T_x)
    Y -- Étiquettes vraies (d_y, n, T_x)
    learning_rate -- Taux d'apprentissage
    epochs -- Nombre d'itérations d'entraînement
    """

```

```

n_samples = X.shape[1]
h0 = np.zeros((self.d_h, n_samples)) # Initialisation de l'état caché

for epoch in range(epochs):
    # Propagation avant
    h, y_pred, caches = self.gru_forward(X, h0)

    # Calcul de la perte
    loss = self.compute_loss(y_pred[:, :, -1], Y[:, :, -1])

    # Initialisation des gradients
    dh = np.zeros_like(h)
    # Gradient au dernier pas de temps
    dh[:, :, -1] = np.dot(self.parameters['Wy'].T, (y_pred[:, :, -1] - Y[:, :, -1]))

    # Rétropropagation
    gradients = self.gru_backward(dh, caches)

    # Mise à jour des paramètres
    for param in gradients:
        if param.startswith('dw') or param.startswith('db'):
            self.parameters[param[1:]] -= learning_rate * gradients[param]

    # Affichage périodique
    if epoch % 10 == 0:
        print(f"Epoch {epoch} Perte: {loss:.4f}")

def predict(self, X):
    """
    Fait des prédictions avec le GRU entraîné.

    Paramètres :
    X -- Données d'entrée (d_x, n, T_x)

    Retourne :
    y_pred -- Prédictions (d_y, n, T_x)
    """
    _, y_pred, _ = self.gru_forward(X, np.zeros((self.d_h, X.shape[1])))
    return y_pred

```

```

In [5]: # Test du code
# Création d'un jeu de données simple
def create_dataset(corpus, seq_length=3):
    """Convertit un corpus en séquences d'entraînement"""
    X, Y = [], []
    for word in corpus:
        for i in range(len(word) - seq_length):
            seq = [char_to_idx[c] for c in word[i:i + seq_length]]
            target = char_to_idx[word[i + seq_length]]
            X.append(seq)
            Y.append(target)

    # Encodage one-hot
    vocab_size = len(chars)
    X_onehot = np.zeros((len(X), vocab_size, seq_length))
    Y_onehot = np.zeros((len(X), vocab_size, 1)) # Un seul caractère cible

    for i, seq in enumerate(X):
        for t, idx in enumerate(seq):
            X_onehot[i, idx, t] = 1
            Y_onehot[i, Y[i], 0] = 1

    return X_onehot.transpose(1, 0, 2), Y_onehot.transpose(1, 0, 2)

# Hyperparamètres
SEQ_LENGTH = 3
HIDDEN_DIM = 20
EPOCHS = 100

# Préparation des données
X, Y = create_dataset(corpus, seq_length=SEQ_LENGTH)

# Initialisation du modèle
model = GRU(input_dim=len(chars), hidden_dim=HIDDEN_DIM, output_dim=len(chars))

# Entraînement
model.train(X, Y, learning_rate=0.01, epochs=EPOCHS)

# Fonction de prédiction
def predict_next_char(model, seed_seq):
    """Prédit le prochain caractère à partir d'une séquence"""
    seed_idx = [char_to_idx[c] for c in seed_seq]
    x = np.zeros((len(chars), 1, len(seed_seq)))
    for t, idx in enumerate(seed_idx):
        x[idx, 0, t] = 1
    y_pred = model.predict(x)
    return idx_to_char[np.argmax(y_pred[:, 0, -1])]

# Test de prédiction
print(predict_next_char(model, "bonjou")) # Devrait prédire 'r'

Epoch 0 Perte: 2.9444
Epoch 10 Perte: 2.9443
Epoch 20 Perte: 2.9441
Epoch 30 Perte: 2.9440
Epoch 40 Perte: 2.9439
Epoch 50 Perte: 2.9438
Epoch 60 Perte: 2.9436
Epoch 70 Perte: 2.9435
Epoch 80 Perte: 2.9434
Epoch 90 Perte: 2.9433
u

```

2.4 - Comparaison

- Entraînement des modèles

Nous allons maintenant entraîner les trois modèles sur les données préparées.

- Comparaison des performances

Nous allons comparer les performances des trois modèles en termes de perte et de précision.

